

LangChain 整体总结：AI Agent 第一阶段学习完成 已付费

原创 神说要有光 神光的幸福生活 2026年3月3日 20:26 山东

LangChain 部分学完了，我们来整体总结一下。

首先，我们为什么要用 LangChain 这种 AI Agent 开发框架？

市面上有很多大模型，它们的 api 格式整体分为三类：

- OpenAI
- Anthropic (Claude)
- Google Gemini

国产大模型的 api 都兼容 OpenAI 格式

举个例子，比如 system 消息怎么传。

OpenAI 格式是这样：

```
{
  "model": "gpt-3.5-turbo",
  "messages": [
    {"role": "system", "content": "你是代码助手"},
    {"role": "user", "content": "你好"}
  ]
}
```

放在 messages 数组里。

Anthropic 格式是这样：

```
{
  "model": "claude-4.5-opus",
  "system": "你是一个代码助手",
  "messages": [
    {
      "role": "user",
      "content": [{"type": "text", "text": "分析这段代码"}]
    }
  ]
}
```

放在单独的 system 字段

Gemini 的格式是这样：

```
{
  "contents": [{
    "role": "user",
    "parts": [{ "text": "解释下这段代码" }]
  }],
  "system_instruction": { // 系统指令又是另一种写法
    "parts": [{ "text": "你是一个代码专家" }]
  }
}
```

放在 system_instruction 字段。

类似这样的差异挺多，如果直接和具体大模型耦合，那你的代码就没法切换其他模型了。

所以需要有一个统一的写法，然后适配不同的大模型。

你如果用 LangChain，就是这样：

所有大模型的 api 都实现 BaseChatModel

这样调用的时候，api 一样。

细节由 ChatXxx 去实现。

它们在不同的包里：

<https://www.npmjs.com/package/@langchain/google-genai>

<https://www.npmjs.com/package/@langchain/deepseek>

<https://www.npmjs.com/package/@langchain/anthropic>

也可能是在 @langchain/community 包：

神光的编程秘籍

有同学说，不对啊，不是说国产大模型都支持 OpenAI 格式么？之前我们也是直接用 ChatOpenAI 调用的 qwen-plus 之类的模型，为啥还有单独的 ChatModel

是的，虽然这些模型兼容 OpenAI 格式，但是每个大模型都有一些自己独有的细节，如果想用全部特性，还是要用专门的 ChatModel 类。

所以，为什么要用 LangChain？

它可以用统一的 ChatModel api 来调用各种大模型，屏蔽了底层差异。

比如我司项目里就用到了各种大模型：

基于 LangChain 可以做到切换各种大模型，代码不变。

所以，我们是基于 Langchain 的 api 来学习大模型的特性，而不是直接学某个大模型的特定 api。

通过 BaseChatModel 屏蔽了大模型底层差异后，再就是对输入、输出做控制：

这就用到了 PromptTemplate 和 OutputParser 的 api

我们基于 prompt 来调用大模型

prompt 可能会很复杂，而且会长期迭代，这就需要组件化管理，用的时候组合，而且 prompt 里还需要加少量案例（Few Shot）。

所以 LangChain 提供了 PromptTemplate 的 api。

通过 ChatPromptTemplate 创建 prompt 模板：

其中的占位符用的时候传入。

如果是对话记录，是通过 MessagePlaceholder 传入：

多个 PromptTemplate 可以用 PipelinePromptTemplate 组合：

比如指定多个 pipelinePrompts，然后指定最终的 finalPrompt

这样就是多个 PromptTemplate 合成一个。

有时还需要加入一些示例，用 FewShotPromptTemplate

指定模板和填入的值就可以了：

生成的就是这种带少量示例（Few Shot）的 prompt：

而且还可以根据长度、语义来做示例选择

这些就是 Prompt Template 的核心 api 了

然后是输出部分，也就是 OutputParsr：

我们希望大模型按照我们指定的格式输出，比如某个 json 结构。

这依赖两种机制：tool_call、json schema

大模型训练的时候就强制 tool_call、json schema 只能输出符合格式的 json。

所以能保证返回的一定是符合格式要求的。

如果都不支持，也可以用 OutputParser，也就是在 prompt 里带上格式要求，然后按照这个格式解析：

但不用自己区分用哪种方式，直接调用 `model.withStructuredOutput` 就可以了，LangChain 会根据调用的模型来选择用哪种（优先 tool call）。

我们做了一个智能录入数据的例子：

AI 应用里这个功能很常见。

一般用 `model.withStructuredOutput` 就可以了，但在一些场景下，还是需要 `OutputParser` 的：

- 流式打印
- 非 json 格式

比如我们做的流式版 mini cursor

神光的编程秘籍

就是用的 `JsonOutputToolsParser` 解析了流式的内容：

之前流式返回的内容是这样：

参数片段在 `tool_call_chunks` 里

用了 `JsonOutputToolsParser` 是这样的：

会解析成 json 格式。

这时候的片段信息不完整，比如少了大括号，少了一半引号等

如果自己解析 json 还是挺麻烦的，就可以直接用这个 `OutputParser`

类似这种 `OutputParser` 我们也学了一些：

- **`StringOutputParser`**：从各种格式里取出内容，返回字符串
- **`StructuredOutputParser`**：按照某种 JSON 格式返回内容并解析成对象
- **`XMLOutputParser`**：按照 xml 格式返回内容并解析成对象
- **`JsonOutputToolsParser`**：解析 `tool_call` 的信息，支持流式

在大模型的输出控制方面，`model.withStructuredOutput` 加上 `OutputParser` 就够用了。

输出格式控制用到了 `tool_call`，这是我们最先学的特性

定义 tool，加一下 name、description、参数 schema

然后 `model.bindTools` 绑定到大模型

只要描述写的清楚，那大模型就会在需要调用 tool 的时候返回 `tool_calls` 信息：

并且按照你指定的 schema 来填充参数。

这样我们根据 `tool_calls` 去调用工具，然后把结果封装成 `ToolMessage` 也放入 `messages` 数组。

之后继续循环调用：

直到没有新的 `tool_call`，循环结束。

当然，不是所有的 `tool` 都要自己写，有很多 MCP，也就是可跨进程调用的 `tool` 可以直接复用

如果 MCP Server 跑在本地进程，就是用 `stdio` 进程通信，否则就是 `http` 通信

比如上面高德 MCP 是用了 http 通信，而 Chrome Devtools 的 MCP 用了 stdio 本地进程通信

在 cursor 等编辑器里配置好 MCP Server 后就可以看到 MCP 提供的所有的 tools

代码里是用 @langchain/mcp-adapters 这个包来和 MCP Server 通信

client.getTools 拿到所有 tool，然后绑定到大模型就好了，其余的和自己定义的 tool 没区别。

依然是这个循环：

这个循环如果调用次数少，没啥问题，把所有对话放到 messages 数组

但是如果聊的多了，这样可能会超过大模型上下文限制，就需要做一些 memory 的处理。

比如你用 cursor、claude code 的时候，token 到了上限就会触发总结：

cursor:

claude code:

当然，messages 数组的写法太原始，一般用 ChatMessageHistory 的 api：

它可以把 messages 存到内存、redis、文件、数据库等。

memory 的管理策略也有三种：

- 截断，去掉之前的一些 message
- 总结，调用大模型对之前的 messages 生成摘要
- 检索，基于向量数据库根据 query 检索之前聊的内容来继续聊

长时记忆基本都是要用向量数据库检索的。

检索涉及到 RAG，这基本也是 Agent 必备的功能。

把一段内容向量化，在坐标空间内就可以通过夹角来判断相似度：

也就是余弦相似度。

当然实际上向量的维度很大，比如 1024

基于 Milvus 之类的向量数据库，可以快速根据向量的余弦相似度，检索出相关文档。

RAG 的流程是这样的：

首先内容存入向量数据库：

各种来源的内容，通过 loader 加载，用 Splitter 分割后再用嵌入模型向量化，存到 Milvus 之类的向量数据库。

之后根据 query 向量化之后去做余弦相似度匹配，就可以检索出相关文档，让大模型生成回答：

比如我们做的电子书阅读助手：

就是检索了 5 个片段，然后给大模型基于这些语义相关的片段来生成回答：

这就是 RAG 的流程。

当然，我们是直接用的 @zilliz/milvus2-sdk-node 这个 Milvus 的包

实际上 LangChain 有一层封装，在 @langchain/community 包下：

用这层封装是更好的，就像前面讲 ChatModel 一样，它也是屏蔽了底层差异。

调用 `similaritySearchVectorWithScore` 做相似度检索。

至此，我们 LangChain 的各个组件就都过了一遍：

ChatModel、PromptTemplate、OutputParser、tool、mcp、memory、RAG

但如果硬编码的方式组合这些组件不好管理，每个人写法都不一样。

而且如果你想加一下监测某个组件输入输出、执行耗时、token 消耗等逻辑，也得硬编码。

所以 LangChain 提供了一种声明式的编码方式：LCEL。

LCEL 就是：

每个组件都实现了 Runnable 接口，比如 ChatModel、OutputParser、PromptTemplate 等。

并且提供了一系列 Runnable 的 api 可以连接不同的组件：

这样组装出一条 chain 之后，统一执行。

而且调用方式有 invoke（同步调用）、stream（流式）、batch（批量调用） 三种：

再回到刚才那个问题，有了声明式的 chain 之后，再加耗时、token 消耗、输入输出的日志等，怎么办呢？

这样：

只要加一个 `callbacks` 回调就可以了，所以的节点就动态加上了这段逻辑。

这就是声明式写法的好处。

后面我们会学 `LangSmith`，它是用来做 `chain` 执行的监测的，它是怎么监测每个节点的情况的呢？

就是基于 `Runnable` 的 `callbacks`

也就是说，通过 `LCEL` 的写法，把各个组件用声明式的方式连接起来，可以动态加一些逻辑。

而且每个节点自带了重试、备选方案、配置等功能，开箱即用：

当然，这种写法要学一些 api：

- **RunnableSequence**：顺序执行
- **RunnableLambda**：把函数包装成 Runnable
- **RunnableMap**：并行执行多个 chain，结果放在对象属性上
- **RunnableBranch**：if else 逻辑
- **RouterRunnable**：switch case 逻辑，根据 key 决定执行哪个 chain
- **RunnableEach**：循环数组每个元素来调用 chain
- **RunnablePassthrough**：拿到原始输入
- **RunnablePick**：取输入对象的某些属性返回
- **RunnableWithMessageHistory**：给 chain 加上 memory

刚开始可能不大习惯，但是多练习写几个 chain 就会了。

(写法上可以简化，函数会自动转成 RunnableLambda、对象会自动转成 RunnableMap)

基本都是这三步：

- 分析流程，拆分原子步骤
- 根据步骤之间的关系，选择对应 Runnable api
- 统一调用 (invoke、stream、batch)

总之，有了 LCEL 后，LangChain 就不再只是工具集，而是一个工业化流水线

每个节点都自带一些功能，还可以给每个节点动态加一些逻辑。

这些就是 LangChain 的全部功能了：

- 各个组件
- LCEL 连接组件成为 chain

我们的学习也是从这两方面来学的。

总结

LangChain 通过 ChatModel 屏蔽了各种大模型的差异，可以用同样的 api 来写代码，可以切换大模型。

我们过了一遍各种组件 ChatModel、PromptTemplate、tool & mcp、OutputParser、memory、RAG 等

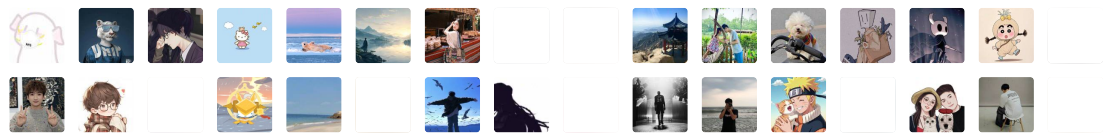
然后是 LCEL 组合各种组件，编排 chain，它可以给节点动态增删逻辑，而且还内置了一些功能。

学完组件可以说 LangChain 是工具集，学完 LCEL 就可以说 LangChain 是工业流水线了。

把这两方面都掌握好，LangChain 就学的差不多了。

学到这里，AI Agent 学习整体告一段落，你也可以自己总结回顾下了。

2774 人付费



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

实战练习 LCEL 组装 chain

下一篇

Nest + LangChain 实现基于 SSE 的流式 ai 接口

